

LDC (LONGITUDINAL REDUNDENCY CODE)

2 boyutlu VRC gibi. Byteleri kolon kolon dize.

Hem yatayda hem dikeyde parity check yapar.

Gift yönlü hata değişimini algılayamadığı senaryolar olabilir.

Hata kontrolü için eklediğimiz bit miktarı yüksektir

	Byte 1		Byte 2		Byte 3		Byte 4		LRC
	1	→	0	→	1	→	1	→	1
	0	↓	0		1		1		0
	0	↓	1		0		1		0
	1	↓	1		0		1		1
	1	↓	0		1		0		0
	0	↓	1		1		0		0
	1	↓	0		0		0		1
VRC	0		1		0		0		1

yatay
parity check cifte tamamlama

10011010	00110101	11001100	11110000	10010011
----------	----------	----------	----------	----------

10011010	01110111	11001100	10110010	10010011
----------	----------	----------	----------	----------

Data bozuldu ama kontrol blokları değişmedi. Tespit edilemeyen hata tiplerine örnek.

Hata olursa tüm frame tekrar gönderilir.

CRC (CYCLIC REDUNDENCY CHECK)

Günümüzde kullanılır.

- 1) Asel polinomlar kullanılır. Gönderilecek veri bu polinomlara bölünür.
- 2) Karşı tarafa bu bölümden kalanı CRC verisi olarak göndereceğiz.
- 3) Karşı taraf bölmeyi kendi de yapar.
- 4) Bulduğu sonuçtan bizim gönderdiğimiz CRC değerini çıkarır.
- 5) Sonuç 0 ise hata yoktur. Değilse vardır.

Genelde kullanılır CRC kontrol polinomları 13. 17. ve 33. dereceden polinomlar.

Tespit edilememe ihtimali yok.

Commonly used polynomials in CRC technique:

- CRC-12 $x^{12}+x^{11}+x^3+x+1$
- CRC-16 $x^{16}+x^{15}+x^2+1$
- CRC-ITU $x^{16}+x^{12}+x^5+1$
- CRC-32 $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$

ör data: 100100 ve kullanılacak polinom: $x^3 + x^2 + 1$ ise CRC = ?

- 1) Önce polinomun binary karşılığı bulunur.
Her terim için;
Kuvveti olanlar için 1,
olmayanlar için 0 yazılır.

$$\underbrace{x^3 + x^2 + 0 \cdot x^1 + 1}_{m \text{ adet bit}}$$

- 2) Verinin sonuna m-1 tane sıfır yerleştiririz.

$$\underbrace{100100}_{\text{data}} \underbrace{000}_{m-1}$$

- 3) Bölme işlemi, polinom bölmesi gibi yapılır.

$$\begin{array}{r} \text{data} \quad m-1 \quad m \\ 100100000 \quad 1101 \\ \oplus 1101 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 01000 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \oplus 1101 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 01010 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \oplus 1101 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 01110 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \oplus 1101 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 00110 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \oplus 0000 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 01100 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \oplus 1101 \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 0001 \end{array}$$

bölünenin en solundaki bit 1' ise vardır
diyip, bölüne 1 ekleyip XOR işlemi yaparız.

n dereceden bir polinomu (n+1) bitle
ifade ederiz. giden dataya ise n+1-1
yani n bitlik bir CRC kodu eklenir.

m-1 bit CRC kodumuz.

CHECKSUM

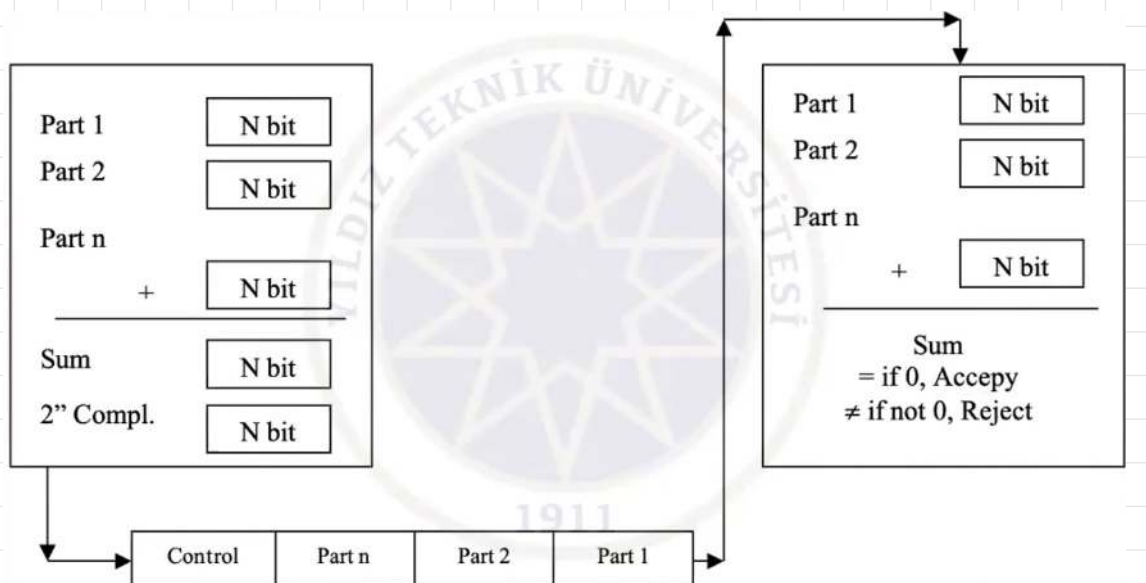
- 1) veriyi önceden belirli boyutta parçalara bölünür. (genelde 16 bitlik)
- 2) her biri için 1's complement alınır.
- 3) first complementler toplanır, toplamın 2's complementi alınır.
- 4) bu hesaplanan değer checksum olarak veriye eklenir.

yine tespit edilemeyen hatalar olabilir. tek sayılı hatalarda daha başarılı.

ör 256 bitlik data = 16x16 bitlik bloklar

$$\begin{array}{l} 16 \text{ bit} \rightarrow 1's \text{ complement} \\ 16 \text{ bit} \rightarrow 1's \text{ complement} \\ \vdots \\ 16 \text{ bit} \rightarrow 1's \text{ complement} \\ + \\ \hline 16 \text{ bitlik veri} \\ \downarrow \\ 2's \text{ complement} \\ \text{checksum} \end{array}$$

frame = 256 bit + 16 bit checksum



ERROR CORRECTION

2 yöntem var.

- 1) Veriyi tekrar gönder.
- 2) Bir bitlik hata varsa, **hamming code / distance**

Hamming Code

Tek bitlik hata için konuşursak;

- m bitlik veride, m bitte hata olabilir + hata olmayabilir = $m+1$ durum olma olasılığı var.
- Kontrol bloku uzunluğu $r \geq \log_2(m+1)$ olmalı.
ör: 7 bit veri için $\log_2(7+1) = 3$ bitlik kontrol bloku.

- $m+r$ bit gönderilecek. bu durumda verimiz $m+r$ bit oldu.

yeniden kontrol bloku hesaplırsak $r \geq \log_2(m+r+1)$

ör: 7 bitlik veri, 3 bitlik kontrol bloku için $\log_2(7+3+1) = 4$ bitlik kontrol bloku.

- Kontrol blokları 2'nin kuvvetlerine denk gelen bitlere eklenir.

ör: 1., 2., 4., 8., 16. bitlere.

B ₁₁	B ₁₀	B ₉	B ₈	B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
D ₇	D ₆	D ₅	R ₄	D ₄	D ₃	D ₂	R ₃	D ₁	R ₂	R ₁

- Kontrol bitleri aşağıdaki şekillerde hesaplanır.

Ⓐ

$$\begin{aligned} R_1 &= B_1 \oplus B_3 \oplus B_5 \oplus B_7 \oplus B_9 \oplus B_{11} \\ R_2 &= B_2 \oplus B_3 \oplus B_6 \oplus B_7 \oplus B_{10} \oplus B_{11} \\ R_3 &= B_4 \oplus B_5 \oplus B_6 \oplus B_7 \\ R_4 &= B_8 \oplus B_9 \oplus B_{10} \oplus B_{11} \end{aligned}$$

kendine depend eden değerlerin hesaplanabilmesi için, öncelikle 0 alanlar boş bırakılır

B ₁₁	B ₁₀	B ₉	B ₈	B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
1	0	0		1	1	0		1		

Boş bırakıldıktan sonra yukarıdaki hesaplamalar yapılır. Boş bırakmak almamak demek.

Ⓑ

$$\begin{aligned} R_1 &= B_3 \oplus B_5 \oplus B_7 \oplus B_9 \oplus B_{11} = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1 \\ R_2 &= B_3 \oplus B_6 \oplus B_7 \oplus B_{10} \oplus B_{11} = 1 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 0 \\ R_3 &= B_5 \oplus B_6 \oplus B_7 = 0 \oplus 1 \oplus 1 = 0 \\ R_4 &= B_9 \oplus B_{10} \oplus B_{11} = 0 \oplus 0 \oplus 1 = 1 \end{aligned}$$

B ₁₁	B ₁₀	B ₉	B ₈	B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁
1	0	0	1	1	1	0	0	1	0	1

Karşıya gidecek data.

- Karşı taraf da aynı hesabı tekrar yapar. Bu sefer tüm alanlar doludur.

	R ₄	R ₃	R ₂	R ₁	Info
0	0	0	0	0	Error-free
1	0	0	0	1	1. bit error
2	0	0	1	0	2. bit error
3	0	0	1	1	3. bit error
4	0	1	0	0	4. bit error
5	0	1	0	1	5. bit error
6	0	1	1	0	6. bit error
7	0	1	1	1	7. bit error
8	1	0	0	0	8. bit error
9	1	0	0	1	9. bit error
10	1	0	1	0	10. bit error
11	1	0	1	1	11. bit error

Kendinde bir kontrol biti dizisi oluşur.

Kendindekiyle gelen kontrol blokunu XOR işlemine tabi tutar.

Hata yoksa ikisi aynı olacağından sonuç 0000 olur.

Hata varsa hangi bitte hata olduğunu sonuç binary olarak verir.

Yandaki tablodaki gibi.

Aslında alınan tarafta doğrudan (A) görselinde gösterilen hesaplama yapıp, doğrudan kontrol bitlerinin olduğu gözle bakılırsa, ekstra bir XOR yapmaya gerek kalmadan doğrudan kontrol bitlerine bakılabilir. Çünkü:

Her R_i için; $R_i =$ Gönderen tarafta Hesaplanan R_i XOR Gönderen tarafta Hesaplanan R_i

$$R_i = B_3 \oplus B_5 \oplus B_7 \oplus B_9 \oplus B_{11}$$

$$R_i$$

şekline gelir. Çünkü gönderen taraf (B) tablosunu kullanıp orijinal R_i 'yi hesapladı. Alan taraf ise (A) tablosunu kullanarak orijinal R_i 'yi kendisiyle XOR'lamış oldu. Hata yoksa bir şeyin kendisiyle XOR'lanması 0 vereceğinden kontrol yapılmış olur.

- Hata varsa, verinin tekrar istenmesi, alınan mesajın alındı bilgisinin iletilmesi.
- Başlangıçta mesaj frame'lere ayrılır, frame'lere senkronizasyon için eklemeler yapılır.
- Akış kontrolü burada yapılır. Alıcı göndericinin gönderdiği hızı yetisemiyorsa; bu frekansın ayarlanması sağlanır. (Flow Control)
- Hata kontrolleri, yeniden gönderme / gönderim isteği. (Error Control / Retransmission)
- Paketin doğru adrese teslim edilmesi için, adresin hesaplanması. (Addressing)
- Bağlantı yönetimi: Mesaj göndericem müsait misin? (Line discipline / Link Management)
evet. data paketi yollarım, alındı gönderir. paketler bitince bağlantı bitirilir.

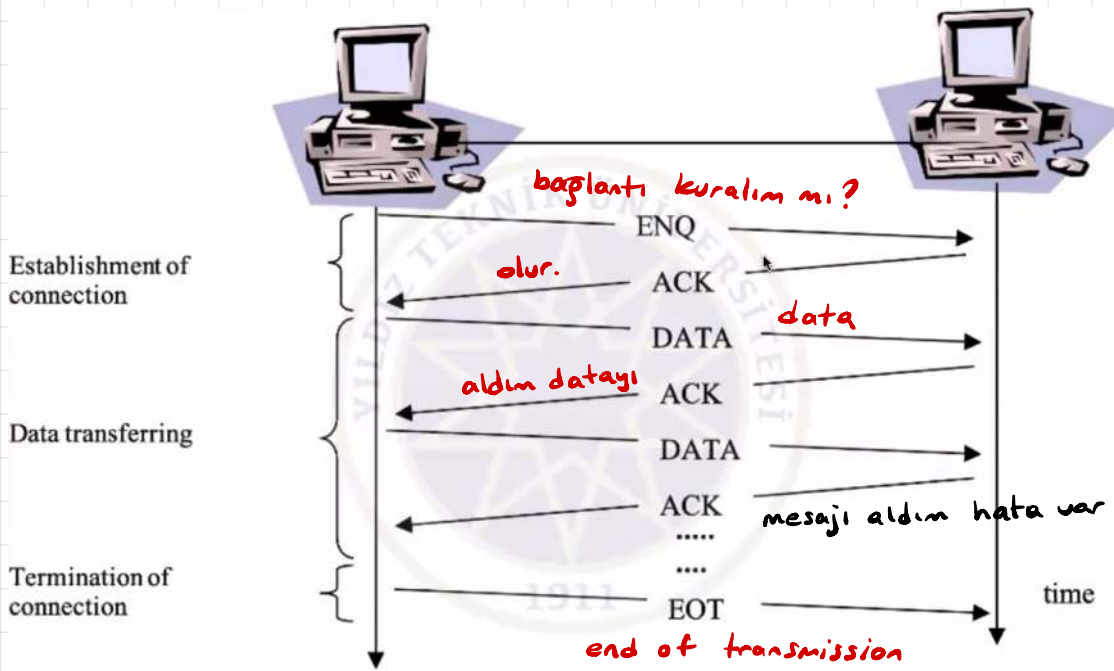
BAĞLANTI YÖNETİMİ

1) Enq/Ack (Enquiry / Acknowledgement) Point-to-Point (WAN'larda)

Point-to-Point: Sadece iki makina arasında gerçekleşir.

Bütün kontroller bu makinelerde yapılır. 3. bir koordinasyon edici cihaz yok.

Birimlerin es özellikte olması gerekir. Ör: ikisi de bilgisayar olmalı.



ENQ: iletişime geçelim mi?

ACK: olur kuralım

NO-ACK: olmaz. yoğun olabilir. baskısıyla bağlı olabilir.
→ bağlantı ok

DATA: frame gönderdik

ACK: aldım, hata yok.

NACK: aldım, hata var ya da almadım.

EOT: bitti.

2) Poll/Select Connection Management Primary Device Aracılığıyla (LAN'larda)

Multipoint.

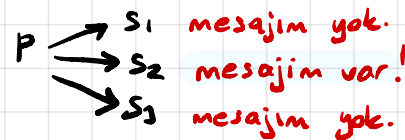
Genelde LAN içinde. Tek hat var. Aynı anda tek iletişim olabilir.

Primary ve secondary cihazlar olur.

Secondary bir cihaz, başka bir secondary ile iletişim kurmak için;

primary'nin kendisine sıra vermesini bekler.

$S_1 \rightarrow P \rightarrow S_2$ gibi. $S_2 \rightarrow P \rightarrow S_1$



Primary konuşmak isterse doğrudan select yapar. Primary konuşurken hat ona ayrılır.

Primary çok konuştuklarını düşünürse, iletişimi durdurup, tekrar polling yapabilir.

Bu yapı olursa, LAN'larda olur. Olmak zorunda değil.